Go语言的GPM调度器是什么?

相信很多人都听说过Go语言天然支持高并发，原因是内部有 协程 （goroutine）加持，可以在一个进程中启动成千上万个协程。那么，它凭什么做到如此高的并发呢？那就需要先了解什么是并发模型。

并发模型

著名的C++专家Herb Sutter曾经说过“免费的午餐已经终结”。为了让代码运行的更快，单纯依靠更快的硬件已经无法得到满足，我们需要利用多核来挖掘并行的价值，而并发模型的目的就是来告诉你不同执行实体之间是如何协作的。

当然，不同的并发模型的协作方式也不尽相同，常见的并发模型有七种：

- 线程与锁
- 函数式编程
- Clojure之道
- actor
- 通讯顺序进程（CSP）
- 数据级并行
- Lambda架构

而今天，我们只讲与Go语言相关的并发模型 CSP ，感兴趣的同学可以自行查阅书籍《七周七并发模型》。

CSP篇

CSP，全称Communicating Sequential Processes，意为通讯顺序进程，它是七大并发模型中的一种，它的核心观念是将两个并发执行的实体通过通道channel连接起来，所有的消息都通过channel传输。其实CSP概念早在1978年就被东尼·霍尔提出，由于近来Go语言的兴起，CSP又火了起来。
那么CSP与Go语言有什么关系呢？接下来我们来看Go语言对CSP并发模型的实现——GPM调度模型。

GPM调度模型

GPM代表了三个角色，分别是Goroutine、Processor、Machine。

[外链图片转存失败,源站可能有防盗链机制,建议将图片保存下来直接上传(img-Xq3OMpgH-1667189455179)(…/images/17188139512ef9a8.jpg)]

- Goroutine：就是咱们常用的用go关键字创建的执行体，它对应一个结构体g，结构体里保存了goroutine的堆栈信息
- Machine：表示操作系统的线程
- Processor：表示处理器，有了它才能建立G、M的联系

Goroutine

Goroutine就是代码中使用go关键词创建的执行单元，也是大家熟知的有“轻量级线程”之称的协程，协程是不为操作系统所知的，它由编程语言层面实现，上下文切换不需要经过内核态，再加上协程占用的内存空间极小，所以有着非常大的发展潜力。

```
go func() {}()
```

复制代码在Go语言中，Goroutine由一个名为runtime.go的结构体表示，该结构体非常复杂，有40多个成员变量，主要存储执行栈、状态、当前占用的线程、调度相关的数据。还有玩大家很想获取的goroutine标识，但是很抱歉，官方考虑到Go语言的发展，设置成私有了，不给你调用😊。

```go
type g struct {
    stack struct {
        lo uintptr
        hi uintptr
    }                           // 栈内存: [stack.lo, stack.hi)
    stackguard0 uintptr
    stackguard1 uintptr

    _panic       *_panic
    _defer       *_defer
    m            *m             // 当前的 m
```

Goroutine调度相关的数据存储在sched，在协程切换、恢复上下文的时候用到。

```go
type gobuf struct {
    sp   uintptr
    pc   uintptr
    g    guintptr
    ret  sys.Uintreg
    ...
}
```

M就是对应操作系统的线程，最多会有GOMAXPROCS个活跃线程能够正常运行，默认情况下GOMAXPROCS被设置为内核数，假如有四个内核，那么默认就创建四个线程，每一个线程对应一个runtime.m结构体。线程数等于CPU个数的原因是，每个线程分配到一个CPU上就不至于出现线程的上下文切换，可以保证系统开销降到最低。

```go
type m struct {
    g0   *g
    curg *g
    ...
}
```

M里面存了两个比较重要的东西，一个是g0，一个是curg。

- g0：会深度参与运行时的调度过程，比如goroutine的创建、内存分配等
- curg：代表当前正在线程上执行的goroutine。

刚才说P是负责M与G的关联，所以M里面还要存储与P相关的数据。

```go
type m struct {
  ...
    p             puintptr
    nextp         puintptr
    oldp          puintptr
}
```

- p：正在运行代码的处理器
- nextp：暂存的处理器
- oldp：系统调用之前的线程的处理器

Processor

Proccessor负责Machine与Goroutine的连接，它能提供线程需要的上下文环境，也能分配G到它应该去的线程上执行，有了它，每个G都能得到合理的调用，每个线程都不再浑水摸鱼，真是居家必备之良品。

同样的，处理器的数量也是默认按照GOMAXPROCS来设置的，与线程的数量一一对应。

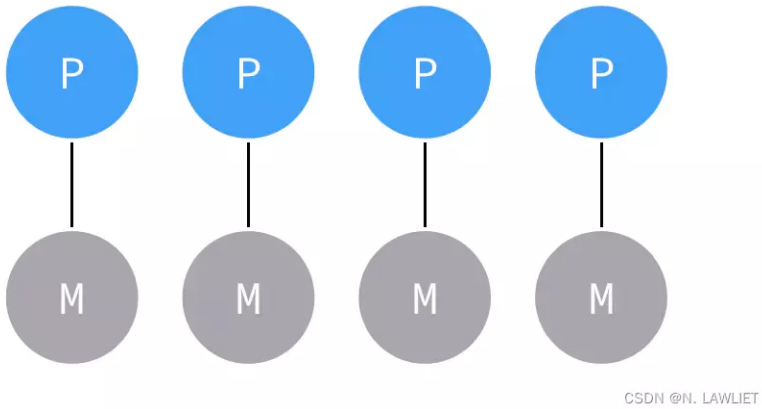
```go
type p struct {
    m           muintptr
```

```
    runqhead uint32
    runqtail uint32
    runq     [256]guintptr
    runnext guintptr
    ...
}
```

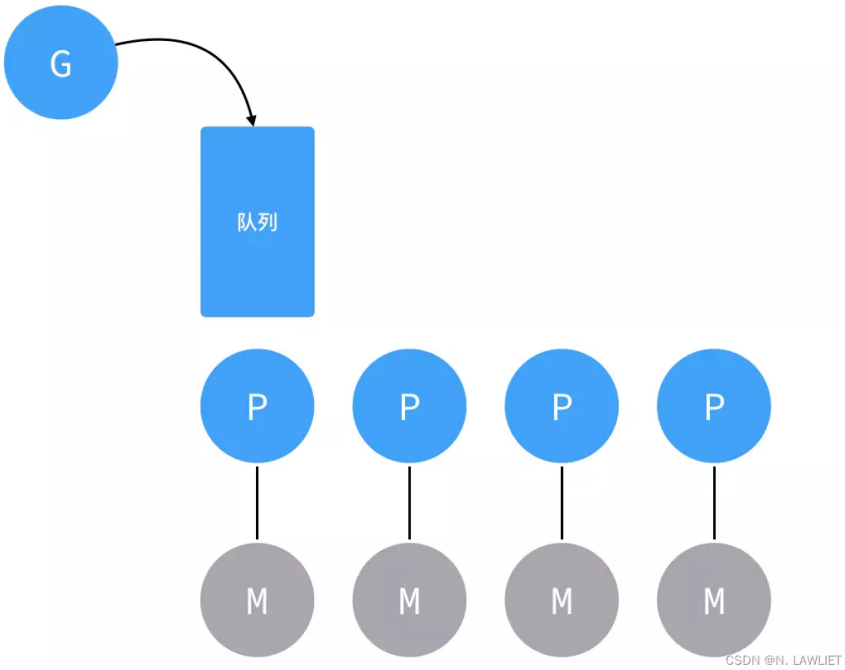
结构体P中存储了性能追踪、垃圾回收 、计时器等相关的字段外，还存储了处理器的待运行队列，队列中存储的是待执行的Goroutine列表。

三者的关系

首先，默认启动四个线程四个处理器，然后互相绑定。



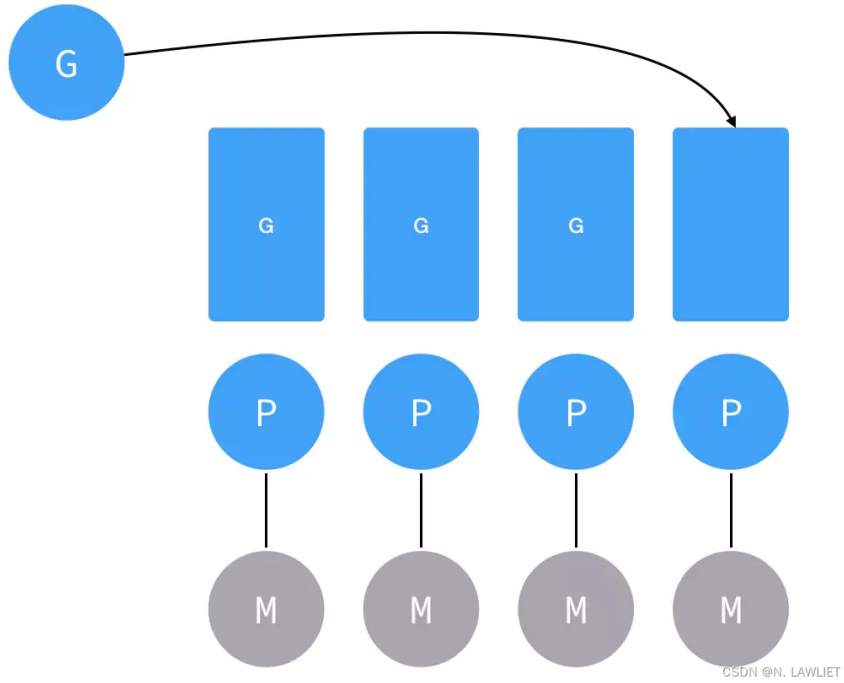
这个时候，一个Goroutine结构体被创建，在进行函数体地址、参数起始地址、参数长度等信息以及调度相关属性更新之后，它就要进到一个处理器的队列等待发车。



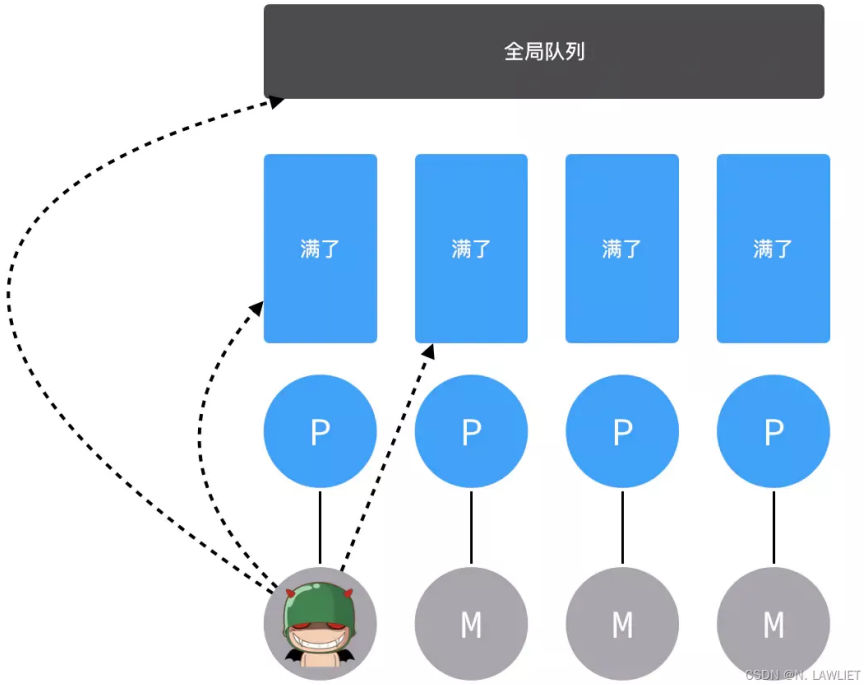
啥，又创建了一个G？那就轮流往其他P里面放呗，相信你排队取号的时候看到其他窗口没人排队也会过去的。

[外链图片转存失败,源站可能有防盗链机制,建议将图片保存下来直接上传(img-wHWY8q06-1667189455186)(H:/资料/interview-go-master/images/1718813a3359a1c0.jpg)]

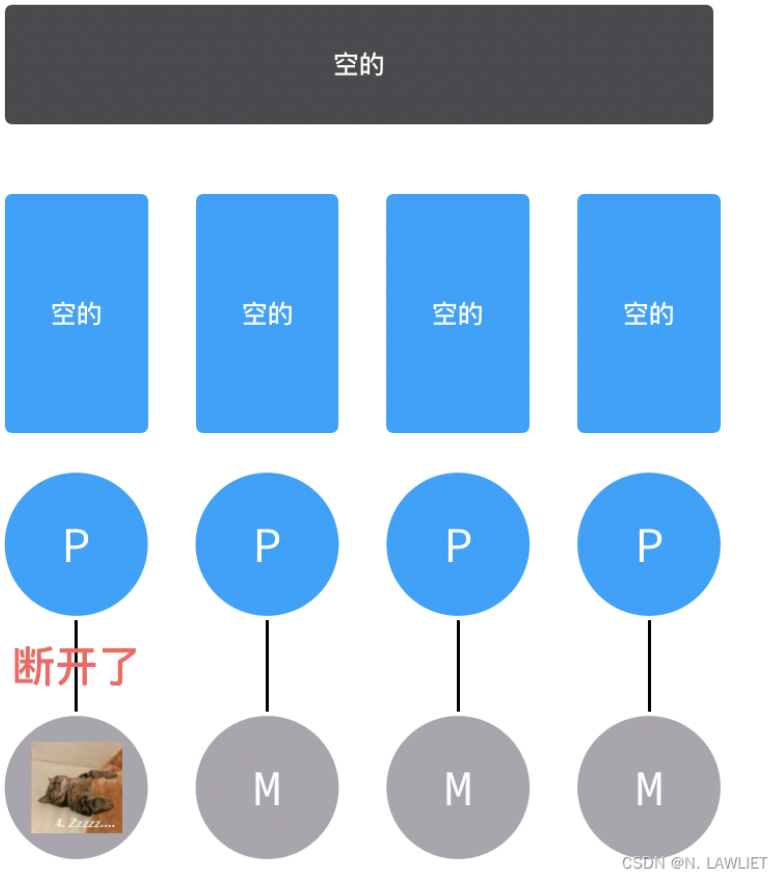
假如有很多G，都塞满了怎么办呢？那就不把G塞到处理器的私有队列里了，而是把它塞到全局队列里（候车大厅）。



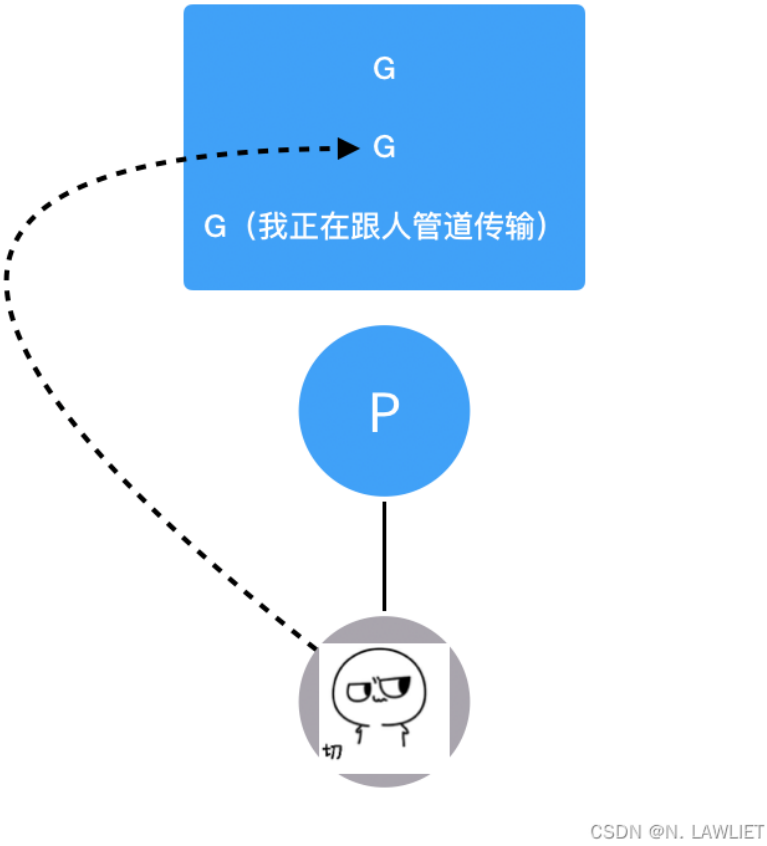
除了往里塞之外，M这边还要疯狂往外取，首先去处理器的私有队列里取G执行，如果取完的话就去全局队列取，如果全局队列里也没有的话，就去其他处理器队列里偷，哇，这么饥渴，简直是恶魔啊！



如果哪里都没找到要执行的G呢？那M就会因为太失望和P断开关系，然后去睡觉（idle）了。

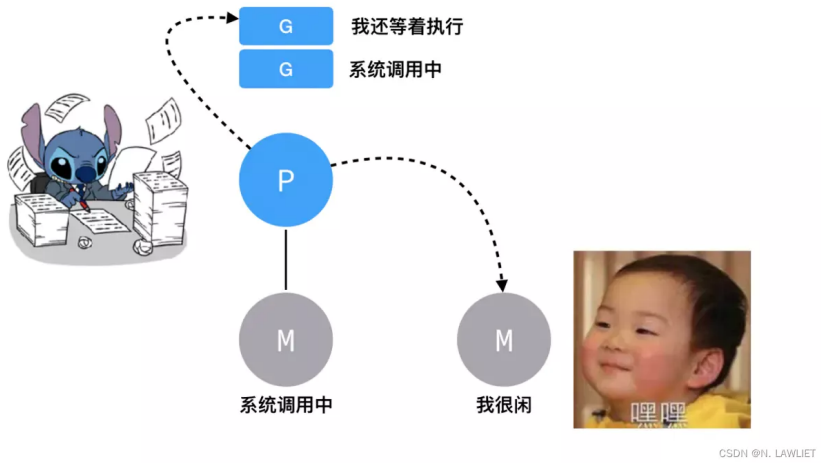


那如果两个Goroutine正在通过channel做一些恩恩爱爱的事阻塞住了怎么办，难道M要等他们完事了再继续执行？显然不会，M并不稀罕这对Go男女，而会转身去找别的G执行。

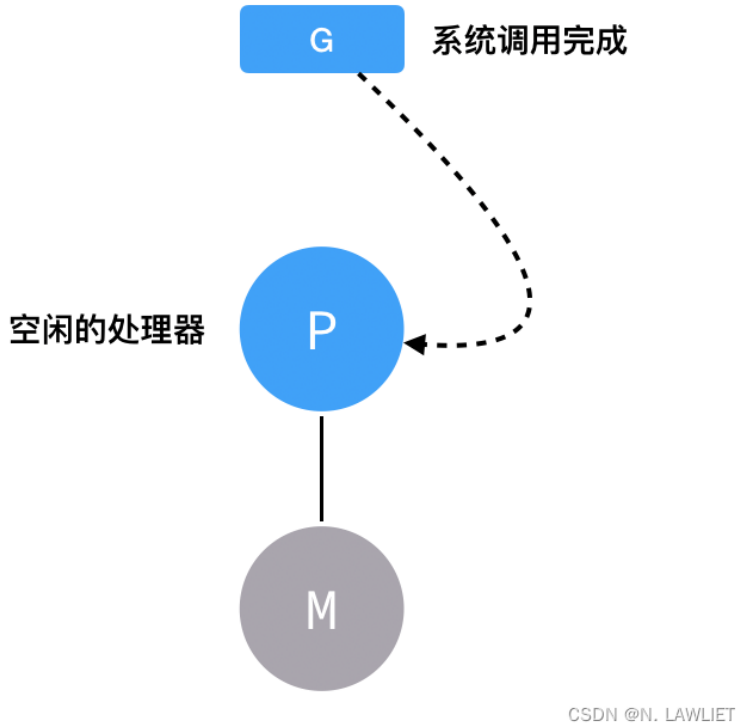


系统调用

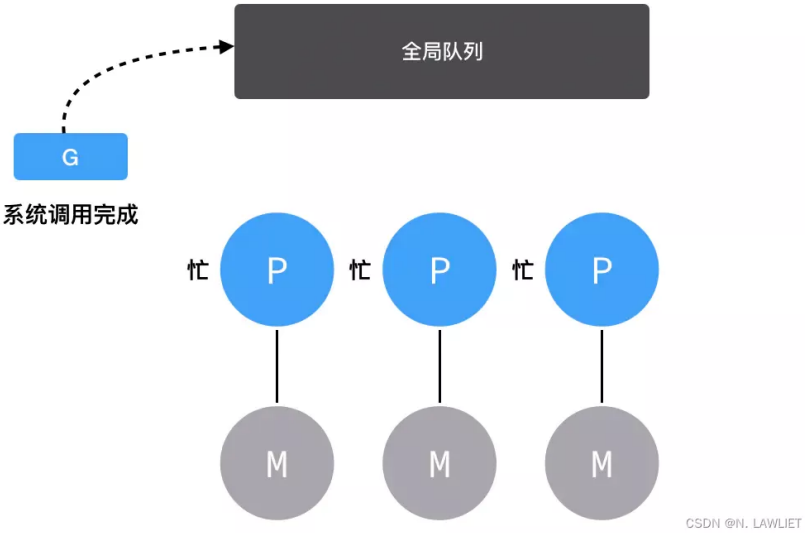
如果G进行了系统调用syscall，M也会跟着进入系统调用状态，那么这个P留在这里就浪费了，怎么办呢？这点精妙之处在于，P不会傻傻的等待G和M系统调用完成，而会去找其他比较闲的M执行其他的G。



当G完成了系统调用，因为要继续往下执行，所以必须要再找一个空闲的处理器发车。



如果没有空闲的处理器了，那就只能把G放回全局队列当中等待分配。



sysmon

sysmon是我们的保洁阿姨，它是一个M，又叫监控线程，不需要P就可以独立运行，每20us~10ms会被唤醒一次出来打扫卫生，主要工作就是回收垃圾、回收长时间系统调度阻塞的P、向长时间运行的G发出抢占调度等等。

目录

Go语言的GPM调度器是什么?
并发模型
CSP篇
GPM调度模型
Goroutine
Processor
三者的关系

系统调用

sysmon